Lesson 1 Stock Enter

This lesson is customized for users who have bought the advanced version.

1. Program Logic

We are going to learn how ArmPi FPV perform intelligent “stock enter” function. The robotic arm can intelligently warehouse the color blocks and blocks with tag. And the whole process consists of three steps, including recognizing, transferring and stock entering.

In the recognition phase, having launched the program, you should turn the bottom holder to make ArmPi FPV search the objects on the map. And the robotic arm will determine the searching result, then it launch corresponding subroutine based on the block searched.

Through tag recognition, the robotic arm can decode the tag within recognition area to corresponding ID number.

The robotic arm target the tag outline through locating, image segmentation and outline searching. Next, it comes to quadrilateral detection, and form a quadrilateral through connecting four corner points. Then, it will code and decode the target tag, and gets corresponding ID.

During color recognition, it will firstly convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

Having finished the previous steps, the robotic arm will move into transferring phase.

In this stage, based on the target block you have set and the processing of

image feedback information, it will adjust the height to pick, pitching angle and lifting height, lifting direction and lifting distance.

Whenever it successfully pick up one block, it will change into intelligent warehousing mode.

According to the shelf set to place the block, the robotic arm will determine the specific placing location on the shelf to finish intelligent warehousing.

The source code is located in /home/armpi_fpv/src/warehouse/scripts/in.py.

```
30  # Warehousing
31  # If not declared, the length and distance units used are all m
32  d_tag_map = 0
33
34  tag_z_min = 0.01
35  tag_z_max = 0.015
36
37  d_color_map = 30
38
39  color_z_min = 0.01
40  color_z_max = 0.015
41  d_color_y = 20
42  color_y_adjust = 400
43
44  center_x = 340
45
46  __isRunning = False
47
48  lock = threading.RLock()
49  ik = ik_transform.ArmIK()
50
51  mask1 = cv2.imread(
    '/home/ubuntu/armpi_fpv/src/object_sorting/scripts/mask1.jpg',
    0)
52  mask2 = cv2.imread(
    '/home/ubuntu/armpi_fpv/src/object_sorting/scripts/mask2.jpg',
```

2. Operation Steps

The entered command must be strictly distinguished between uppercase and lowercase and spaces, and the keywords support the "TAB" key completing.

Let’s take the example of placing the red block on the third shelf on the right and the ID1 block on the third shelf on the left.

2.1 Preparation

① Please place the map on even desk and the robotic arm should be placed on the corresponding area on the map.

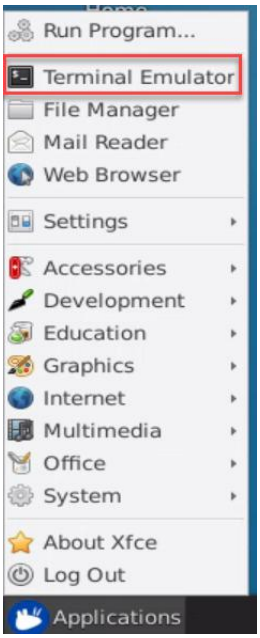
② Prepare 3 blocks in red, green and yellow and 3 blocks labeled with tag1, tag2 and tag3. And place them randomly on the recognition area of the map. Note: the interval between each blocks is not less than 3cm.

③ Turn on the robotic arm and wait to finish.

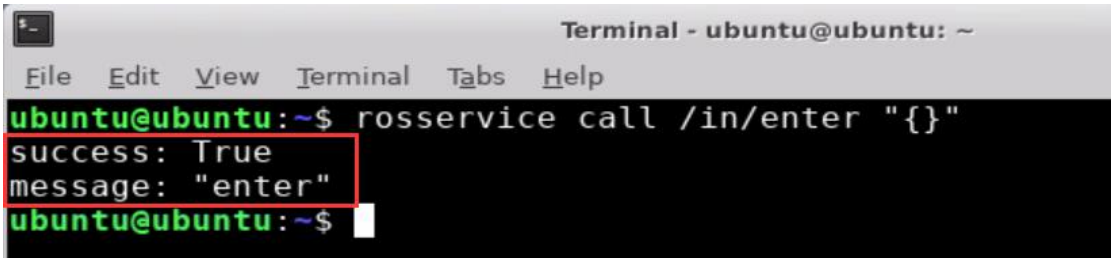
2.2 Enter the Game

① Turn on ArmPi FPV and connect to Raspberry Pi desktop through NoMachine.

② Click “Applications” button and select “ Terminal Emulator” in pop-up interface to open the command line terminal.



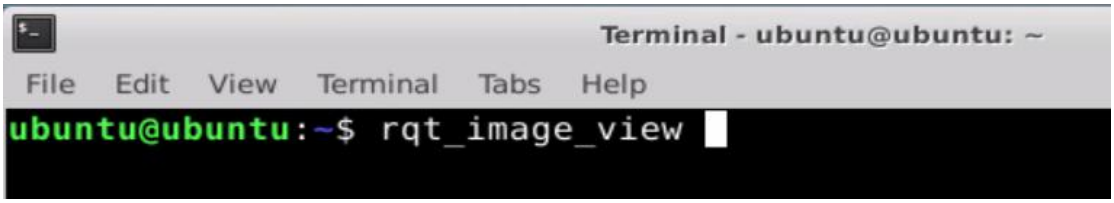
③ Enter **“rosservice call /in/enter "{}""** command, then press “Enter” to start the game. After the game starts, a print prompt will appear, as shown in the figure below.



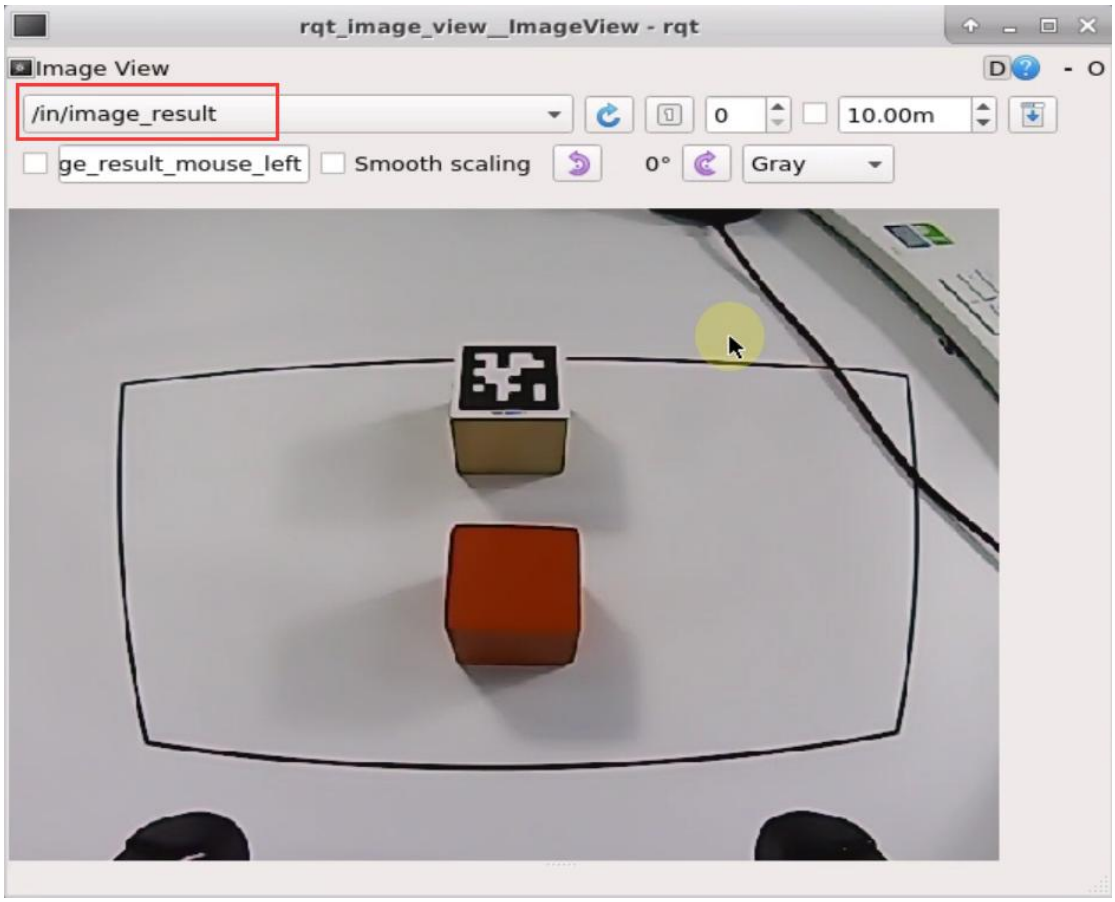
2.3 Turn On the Image

① When entering the game, we need to turn on image returned by camera with rqt. Please go back to the desktop and click the “Application” icon at bottom-left corner to open a new terminal.

② Enter “**rqt_image_view**” command and press “Enter”. Then open rqt after few seconds.



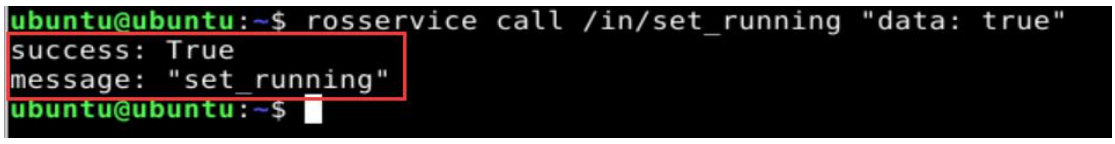
③ As shown in the following figure, select “**/in/image_result**” and other settings remain the same.



Note: after the image is turned on, please be sure to select the topic corresponding to the game you play. Otherwise, the recognition process can not be displayed normally if you start other games.

2.4 Start the Game

① Please go back to the terminal opened in “2.2 Enter the Game” and enter **rosservice call /in/set_running "data: true"** command. When the prompt, shown in the red box, appears, it means you start the game successfully.



② After starting the game, we need to set the parameters including target goods and placing location. Let's take the example of placing the red block on the third shelf on the right and the ID1 block on the third shelf on the left. The entered command is as follow.

```
rosservice call /in/set_target "goods:
- 'red'
- 'tag1'
position:
- 'R3'
- 'L3'"
```

③ There are two types of parameter, and you can refer to the table below for their meanings and built-in parameters.

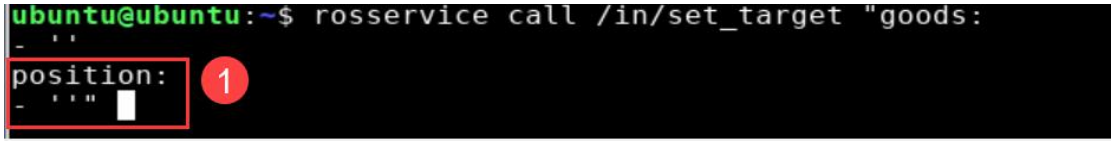
Type	Meaning	Built-in parameter
goods	Object to be warehoused	Tag Block: tag1, tag2, tag3 Color block: red, green, blue
position	Placing Location	Left Shelf (from bottom to top): L1, L2, L3 Right Shelf (from bottom to top): R1, R2, R3

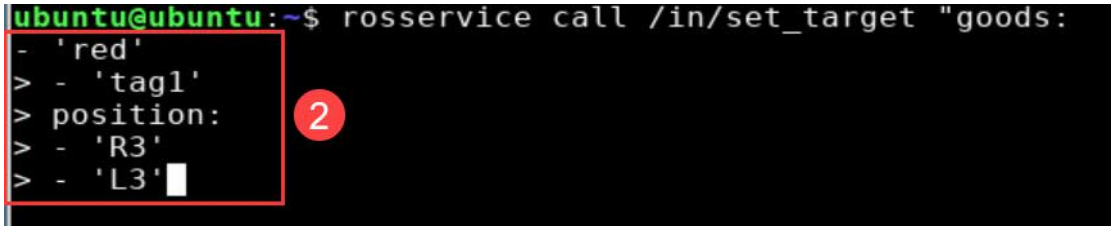
④ Pay attention to the following points when entering parameters.

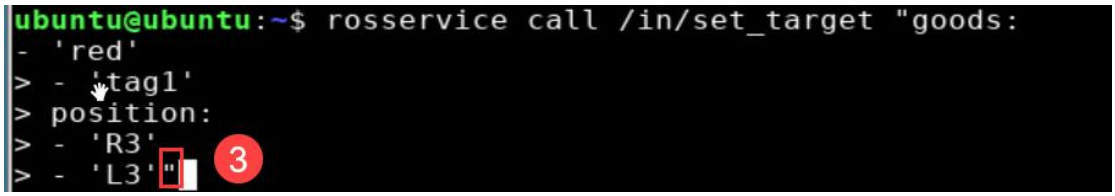
(1) If you are entering multiple parameters (as shown in the figure above), you must wrap between each parameters. And please strictly follow this **format, dash + space + 'shelf number'**.

(2) In order to avoid errors caused by input method, it is recommended to use the Tab key completing, and enter the parameters in the corresponding position.

(3) If you enter the parameters after Tab key completing, then you need to delete the content after the first parameter before wrapping. After all the parameters are entered, add double quotation marks after the last parameter. The instruction is as follow.







(4) There is corresponding relationship between these two types of parameters in terms of location and number, that is, the number of entered position parameter is the same as entered goods parameter. And the robotic arm will place the first target block on the first specific shelf. And the first target block corresponds to first goods parameter and the first specific shelf corresponds to first positions parameter.

2.5 Pause and Quit the Game

You can enter "**rosservice call /in/set_running "data: false""** command

to pause the game.

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_running "data: false"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

Referring to “2.4 Start the game”, you can change and add the blocks to be warehousing and the number of placing shelves.

You can enter “**rosservice call /in/exit "{}"**” command to quit the game.

```
ubuntu@ubuntu:~$ rosservice call /exchange/exit "{}"
success: True
message: "exit"
ubuntu@ubuntu:~$
```

Note: the current game will continue as Raspberry Pi is powered on. To avoid occupying too much RAM, please quit the current game according to the instruction above before playing other AI vision game.

If you want to turn off the image returned by camera, you can turn back to open rqt terminal and press “Ctrl+C”.

3. Function Realization

After recognizing the red block, the robotic arm will pick up the red block and place it on the third layer of right shelf. If tag1 block is recognized, tag1 block will be picked up and placed on the third layer of left shelf. Other blocks can not be recognized.